

Aula 3: APIs RESTful, Modelo de Maturidade de Richardson e CRUD

Bem-vindos à nossa terceira aula! Agora que já temos o nosso projeto Quarkus rodando, é hora de entender como estruturar nossas APIs da maneira correta e colocar a mão na massa criando nossos primeiros endpoints.

Parte 1: API REST e o Modelo de Maturidade de Richardson

Ao construir uma API para a web, o padrão mais utilizado no mercado é o **REST** (Representational State Transfer). Para avaliar o quão "RESTful" (o quão bem segue os princípios REST) uma API é, utilizamos o **Modelo de Maturidade de Richardson**, que divide as APIs em quatro níveis:

- **Nível 0 (O Pântano do POX):** Usa o HTTP apenas como um túnel de transporte. Geralmente tem apenas um endpoint (ex: `/api`) e usa apenas o método POST para enviar comandos variados.
- **Nível 1 (Recursos):** A API começa a introduzir o conceito de "Recursos" (Resources). Em vez de um único endpoint, temos URLs específicas para cada entidade, como `/clientes` ou `/produtos`.
- **Nível 2 (Verbos HTTP):** Aqui é onde a maioria das boas APIs se encontra. Além de ter rotas para recursos, a API utiliza os verbos HTTP corretamente para cada operação: GET para buscar, POST para criar, PUT para atualizar e DELETE para remover. Além disso, utiliza os códigos de status HTTP apropriados (200, 201, 404, etc).
- **Nível 3 (Controles Hiperlinks - HATEOAS):** O nível mais alto. A resposta da API inclui links que indicam aos clientes quais as próximas ações possíveis, tornando a API autodescritiva.

Nesta disciplina, o nosso objetivo prático é construir APIs consistentes que atinjam o **Nível 2**.

Parte 2: Construindo nossa API no Quarkus

Adicionando Dependências (On Demand)

Para criar APIs REST que respondam e recebam dados no formato JSON de forma eficiente, precisamos garantir que as extensões corretas estejam no projeto (como o RESTEasy Reactive e o Jackson). Lembrando o conceito da última aula, podemos adicionar isso sob demanda pelo terminal de forma muito rápida, sem interromper o fluxo de trabalho:

```
▶ mvn quarkus:add-extension -Dextensions="resteasy-reactive-jackson"
```

Controllers (Endpoints) e o CRUD

No Quarkus (que implementa o padrão JAX-RS), geralmente chamamos os "Controllers" de **Resources**. Um *Resource* é uma classe Java que expõe métodos através de rotas HTTP.

Na prática de hoje, vamos construir juntos o **Endpoint de Cliente**, implementando as quatro operações fundamentais de gerenciamento de dados, conhecidas como **CRUD**:

- **Create (Criar):** Usando o verbo `@POST` para adicionar um novo cliente.
- **Read (Ler):** Usando o verbo `@GET` para listar os clientes ou buscar um cliente específico.
- **Update (Atualizar):** Usando o verbo `@PUT` (ou `@PATCH`) para alterar os dados de um cliente existente.
- **Delete (Apagar):** Usando o verbo `@DELETE` para remover um cliente.

Parte 3: Desafio Prático dos Alunos

Após construirmos o CRUD de Clientes passo a passo em conjunto, será a vez de vocês assumirem o controle!

O desafio da parte final da aula será criar de forma autônoma o **Endpoint de Produto**. Vocês deverão aplicar exatamente os mesmos conceitos (estruturação da URL, utilização dos verbos HTTP corretos e manipulação dos dados em memória) para entregar um CRUD completo para a entidade Produto.

Boa leitura e vamos ao código!